

Практическая работа 29. Создание автоматизированных тестов

В этом уроке вы узнаете об автоматизированных тестах в Android и о том, как они позволяют писать масштабируемые и надежные приложения. Вы также узнаете о разнице между логикой пользовательского интерфейса и бизнес-логикой, а также о том, как тестировать и то, и другое. Наконец, вы узнаете, как писать и запускать автоматизированные тесты в Android Studio.

Необходимые условия

- Умение писать приложения для Android с использованием функций и компонент.

Что вы узнаете

- Что делают автоматизированные тесты в Android.
- Почему автоматизированные тесты важны.
- Что такое локальный тест и для чего он нужен.
- Что такое инструментальный тест и для чего он нужен.
- Как писать локальные тесты для кода Android.
- Как писать инструментальные тесты для Android-приложений.
- Как запускать автоматизированные тесты.

Что вы будете создавать

- Локальный тест
- Инструментальный тест

Что вам понадобится

- Последняя версия Android Studio
- Код решения для приложения **Tip Time**

Автоматизированные тесты

Тестирование программного обеспечения - это структурированный метод проверки вашего программного обеспечения, чтобы убедиться, что оно работает так, как ожидается.

Автоматизированное тестирование - это код, который проверяет правильность работы другого написанного вами кода.

Тестирование - важная часть процесса разработки приложений. Постоянно запуская тесты для своего приложения, вы можете убедиться в его корректности, функциональности и удобстве использования до того, как выпустите его в открытый доступ.

Тестирование также дает возможность постоянно проверять существующий код по мере внесения изменений.

Хотя ручное тестирование почти всегда имеет место быть, тестирование в Android часто можно автоматизировать. На протяжении всего остального курса вы сосредоточитесь на автоматизированных тестах для проверки кода приложения и функциональных требований самого приложения. В этом кодебалете вы узнаете самые основы тестирования в Android. В последующих коделабах вы узнаете о более продвинутых практиках тестирования приложений для Android.

По мере знакомства с разработкой Android и тестированием приложений для Android вам следует регулярно писать тесты вместе с кодом приложения. Создание тестов каждый раз, когда вы создаете новую функцию в своем приложении, снижает нагрузку на вас в дальнейшем, по мере роста вашего приложения. Кроме того, это удобный способ убедиться в том, что приложение работает правильно, не тратя много времени на ручное тестирование.

Автоматизированное тестирование является неотъемлемой частью разработки любого программного обеспечения, и разработка Android не является исключением. Поэтому нет лучшего времени для его внедрения, чем сейчас!

Почему автоматизированные тесты важны Сначала может показаться, что тесты в вашем приложении не нужны, но тестирование необходимо в приложениях любого размера и сложности.

Чтобы развивать кодовую базу, вам нужно тестировать существующую функциональность по мере добавления новых элементов, а это возможно только при наличии существующих тестов. По мере роста приложения ручное тестирование требует гораздо больше усилий, чем автоматизированное. Кроме того, когда вы начинаете работать над производственными приложениями, тестирование становится критически важным при наличии большой базы пользователей. Например, вы должны учитывать множество различных типов устройств, работающих под управлением различных версий Android.

В конце концов, вы достигнете точки, когда автоматизированные тесты смогут учитывать большинство сценариев использования значительно быстрее, чем ручные тесты. Проводя тесты перед выпуском нового кода, вы можете внести изменения в существующий код, чтобы избежать выпуска приложения с неожиданным поведением.

Помните, что автоматизированные тесты - это тесты, выполняемые с помощью программного обеспечения, в отличие от ручных тестов, которые выполняются человеком, непосредственно взаимодействующим с устройством. Автоматизированное и ручное тестирование играют важную роль в обеспечении приятного опыта для пользователей вашего продукта. Однако автоматизированные тесты могут быть более точными и оптимизируют производительность вашей команды, поскольку для их запуска не требуется человек, и они могут быть выполнены гораздо быстрее, чем ручные тесты.

Тип автоматизированных тестов Локальные тесты Локальные тесты - это тип автоматизированных тестов, которые непосредственно проверяют небольшой фрагмент кода на предмет его правильного функционирования. С помощью локальных тестов можно тестировать функции, классы и свойства. Локальные тесты выполняются на рабочей станции, что означает, что они работают в среде разработки без использования устройства или эмулятора. Так можно сказать, что локальные тесты выполняются на вашем компьютере. Они также не требуют больших затрат ресурсов компьютера, поэтому могут работать быстро даже при ограниченных ресурсах. Android Studio поставляется с готовыми локальными тестами, которые запускаются автоматически.

Инструментальные тесты Для разработки под Android инструментальные тесты - это тесты пользовательского интерфейса. Тесты инструментов позволяют тестировать части приложения, которые зависят от Android API, а также API и сервисы платформы.

В отличие от локальных тестов, UI-тесты запускают приложение или часть приложения, имитируют взаимодействие с пользователем и проверяют, правильно ли приложение отреагировало. На протяжении всего курса тесты пользовательского интерфейса выполняются на физическом устройстве или эмуляторе.

Когда вы запускаете инструментальный тест на Android, тестовый код фактически встроен в собственный пакет приложений Android (APK), как и обычное приложение Android. APK - это сжатый файл, содержащий весь код и необходимые файлы для запуска приложения на устройстве или эмуляторе. Тестовый APK устанавливается на устройство или эмулятор вместе с APK обычного приложения. Затем тестовый APK запускает свои тесты в сравнении с APK приложения.

Напишите локальный тест

Подготовьте код приложения. Локальные тесты напрямую проверяют методы из кода приложения, поэтому тестируемые методы должны быть доступны для тестирующих классов и методов. Локальный тест в следующем фрагменте кода гарантирует, что метод `calculateTip()` работает правильно, но метод `calculateTip()` в настоящее время является приватным и поэтому недоступен из теста. Удалите обозначение `private` и сделайте его внутренним:

MainActivity.kt

```
internal fun calculateTip(amount: Double, tipPercent: Double = 15.0, roundUp: Boolean): String {
    var tip = tipPercent / 100 * amount
    if (roundUp) {
        tip = kotlin.math.ceil(tip)
    }
    return NumberFormat.getCurrencyInstance().format(tip)
}
```

В файле MainActivity.kt в строке перед методом `calculateTip()` добавьте аннотацию `@VisibleForTesting`:

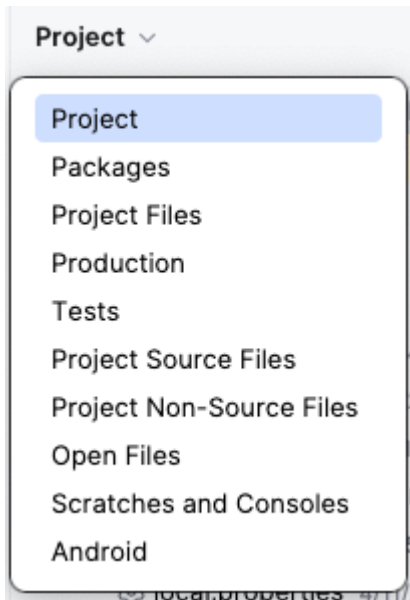
```
@VisibleForTesting
internal fun calculateTip(amount: Double, tipPercent: Double = 15.0, roundUp: Boolean): String {
    var tip = tipPercent / 100 * amount
    if (roundUp) {
        tip = kotlin.math.ceil(tip)
    }
    return NumberFormat.getCurrencyInstance().format(tip)
}
```

Это делает метод общедоступным, но указывает другим, что он общедоступен только в целях тестирования.

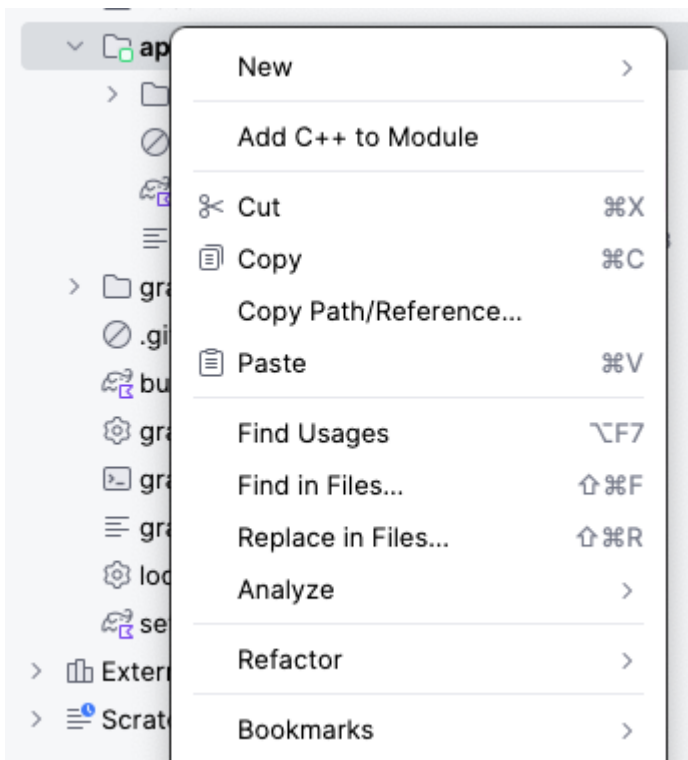
Создание тестового каталога В проектах Android директория test - это место, где пишутся локальные тесты.

Создайте тестовую директорию:

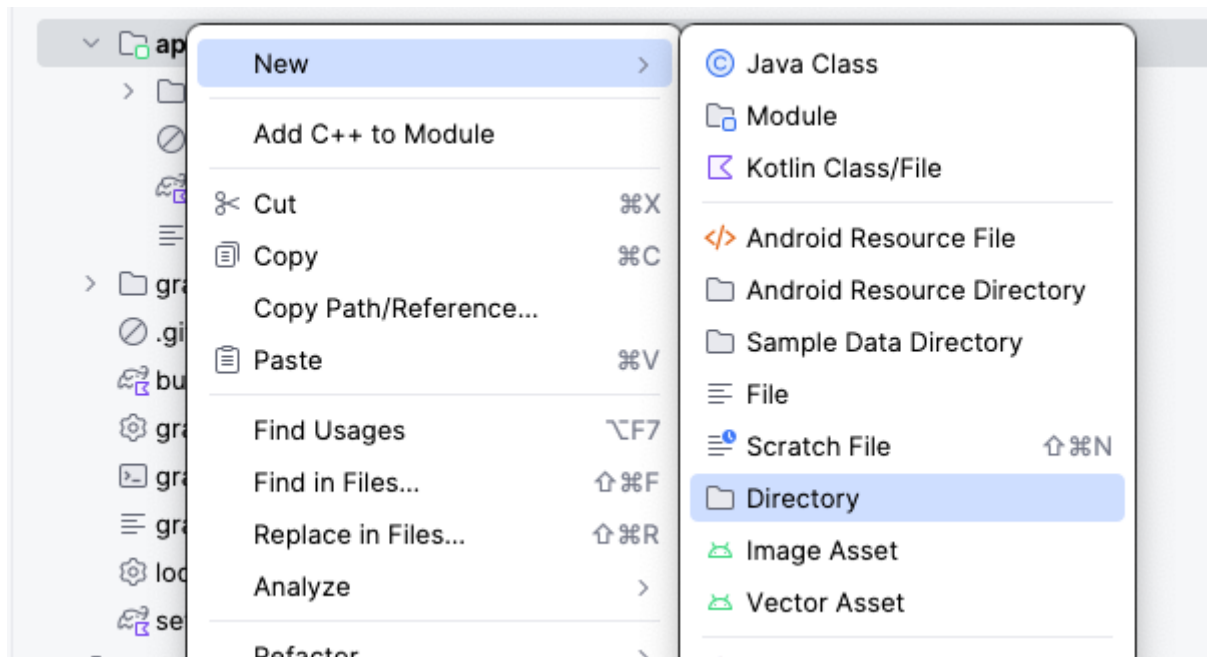
На вкладке Project измените вид на Project.



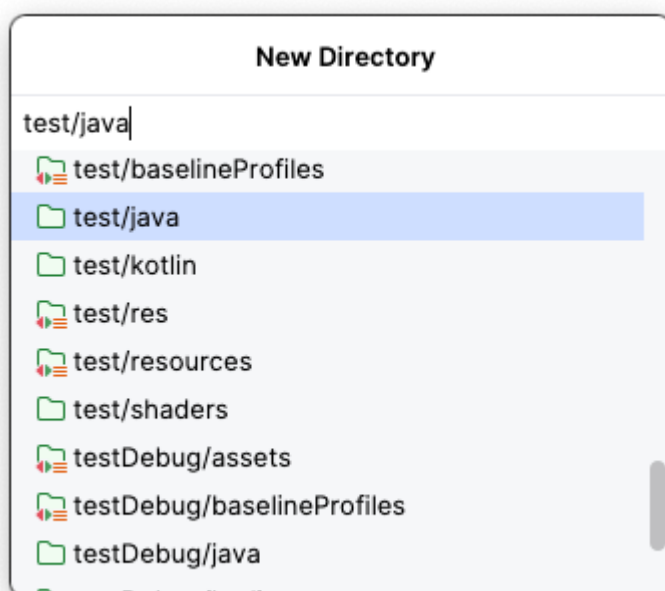
Щелкните правой кнопкой мыши на каталоге src.



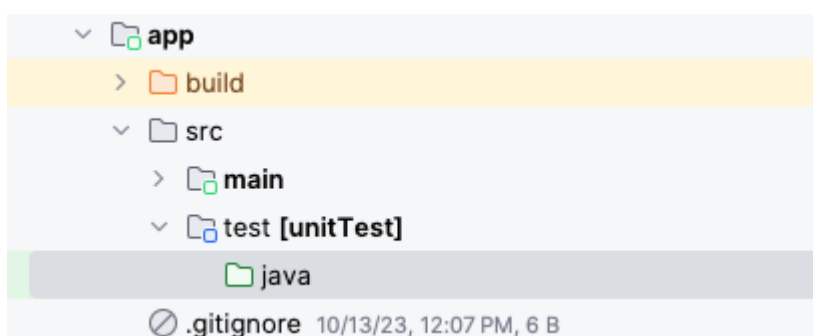
Выберите Новый > Каталог



В окне Новый каталог выберите test/java.



Наберите на клавиатуре клавишу return или enter. Теперь тестовый каталог можно увидеть на вкладке «Проект».

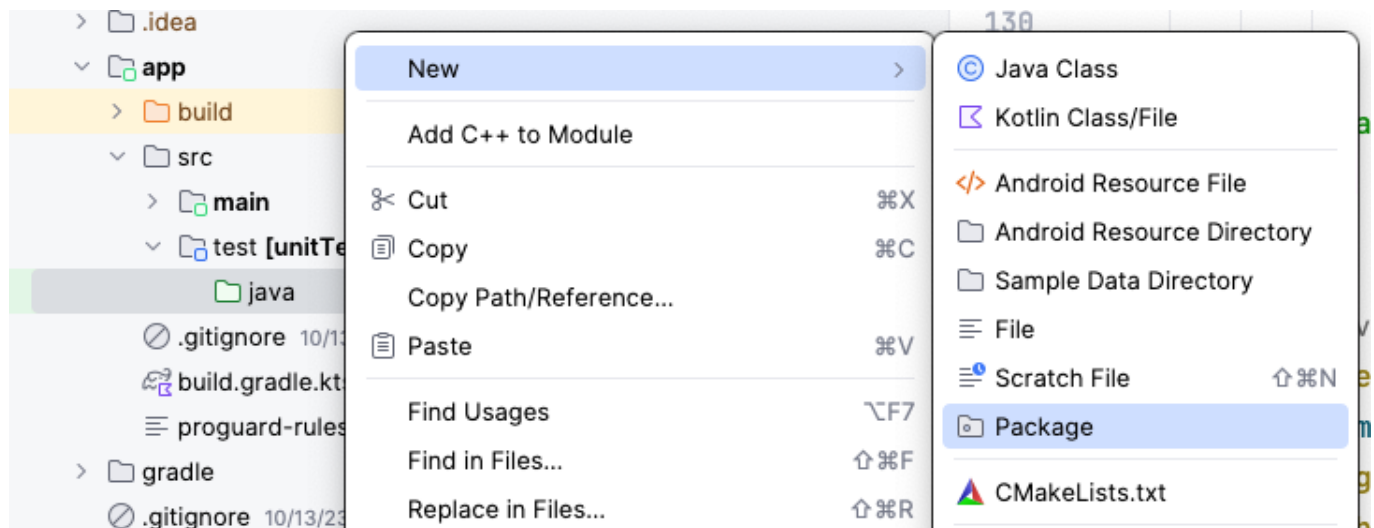


Каталог с тестами должен иметь структуру пакетов, идентичную структуре основного каталога, в котором находится код вашего приложения. Другими словами, точно так же, как код вашего

приложения написан в пакете `main > java > com > example > tiptime`, ваши локальные тесты будут написаны в пакете `test > java > com > example > tiptime`.

Создайте эту структуру пакетов в каталоге `test`:

Щелкните правой кнопкой мыши на каталоге `test/java` и выберите `New > Package`.



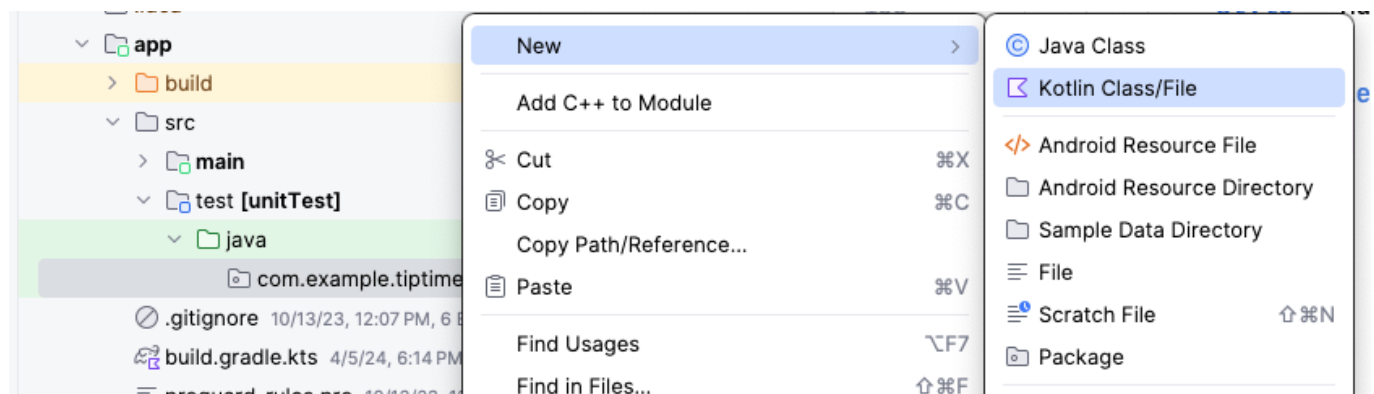
В окне Новый пакет введите `com.example.tiptime`.



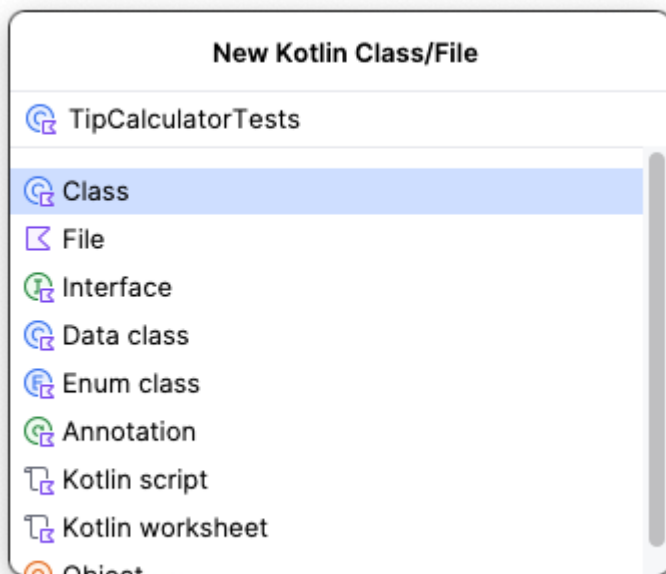
Создайте тестовый класс

Теперь, когда тестовый пакет готов, пришло время написать несколько тестов! Начните с создания тестового класса.

На вкладке Project щелкните `app > src > test`, а затем щелкните стрелку-расширитель `d4706c21930a1ef3.png` рядом с каталогом `test`. Щелкните правой кнопкой мыши каталог `com.example.tiptime` и выберите `New > Kotlin Class/File`.



В качестве имени класса введите `TipCalculatorTests`.



Напишите тест

Как уже говорилось ранее, локальные тесты используются для проверки небольших фрагментов кода в приложении. Основная функция приложения Tip Time рассчитывает чаевые, поэтому необходимо создать локальный тест, который будет гарантировать, что логика расчета чаевых работает правильно.

Для этого нужно напрямую вызвать функцию `calculateTip()`, как это было сделано в коде приложения. Затем вы убедитесь, что значение, возвращаемое функцией, соответствует ожидаемому значению, основанному на значениях, которые вы передали функции.

Есть несколько моментов, которые необходимо знать о написании автоматизированных тестов. Следующий список понятий относится к локальным и инструментальным тестам. Поначалу они могут показаться абстрактными, но к концу этого коделаба вы с ними познакомитесь.

Пишите автоматизированные тесты в виде методов. Аннотируйте метод аннотацией `@Test`. Это позволит компилятору узнать, что метод является тестовым, и запустить его соответствующим образом.

Примечание: Аннотация `@Test` импортируется из пакета `org.junit.Test` с помощью оператора `import org.junit.Test`.

Убедитесь, что название четко описывает, на что проверяется тест и каков ожидаемый результат. Тестовые методы не используют логику, как обычные методы приложений. Их не интересует, как что-то реализовано. Они проверяют только ожидаемый результат при заданных входных данных. Иными словами, тестовые методы выполняют набор инструкций только для того, чтобы убедиться, что пользовательский интерфейс или логика приложения работают правильно. Вам пока не нужно понимать, что это значит, потому что вы увидите, как это выглядит, позже, но помните, что тестовый код может выглядеть совсем не так, как привычный вам код приложения. Тесты обычно заканчиваются утверждением, которое используется для проверки выполнения заданного условия. Утверждения выполняются в виде вызова метода, в названии которого есть `assert`. Например, утверждение `assertTrue()` часто используется при тестировании Android. Утверждения используются в большинстве тестов, но редко применяются в реальном коде приложения. Напишите тест:

Создайте метод для проверки расчета 20% чаевых для суммы счета в \$10. Ожидаемый результат вычисления - \$2.

```
import org.junit.Test

class TipCalculatorTests {

    @Test
    fun calculateTip_20PercentNoRoundup() {

    }

}
```

Возможно, вы помните, что метод `calculateTip()` из файла `MainActivity.kt` в коде приложения требует трех параметров. Сумма счета, процент чаевых и флаг, позволяющий округлить результат или нет.

```
fun calculateTip(amount: Double, tipPercent: Double, roundUp: Boolean)
```

Когда придет время вызвать этот метод из теста, эти параметры должны быть переданы так же, как и при вызове метода в коде приложения.

В методе `calculateTip_20PercentNoRoundup()` создайте две постоянные переменные: переменную `amount`, установленную на значение 10.00, и переменную `tipPercent`, установленную на значение 20.00.

```
val amount = 10.00
val tipPercent = 20.00
```

В коде приложения, в файле `MainActivity.kt`, обратите внимание на следующий код, сумма чаевых форматируется в зависимости от локали устройства.

`MainActivity.kt`

```
...
NumberFormat.getCurrencyInstance().format(tip)
...
```

Такое же форматирование должно использоваться при проверке ожидаемой суммы чаевых в тесте.

Создайте переменную `expectedTip`, установленную в `NumberFormat.getCurrencyInstance().format(2)`. Позже переменная `expectedTip` будет сравниваться с результатом метода `calculateTip()`. Так тест убеждается, что метод работает правильно. На последнем шаге вы установили переменную `amount` в значение 10.00, а переменную `tipPercent` - в значение 20.00. Двадцать процентов от 10 равно 2, поэтому переменная `expectedTip` будет установлена в форматированную валюту со значением 2. Помните, что метод `calculateTip()` возвращает форматированное значение `String`.

Вызовите метод `calculateTip()` с переменными `amount` и `tipPercent` и передайте аргумент `false` для округления. В этом случае вам не нужно учитывать округление, поскольку ожидаемый результат не учитывает округление.

Сохраните результат вызова метода в постоянной переменной `actualTip`. До сих пор написание этого теста мало чем отличалось от написания обычного метода в коде приложения. Однако теперь, когда у вас есть возвращаемое значение из метода, который вы хотите протестировать, вы должны определить, является ли это значение правильным, с помощью утверждения.

Утверждение обычно является конечной целью автоматизированного тестирования и не часто используется в коде приложения. В данном случае вы хотите убедиться, что переменная `actualTip` равна переменной `expectedTip`. Для этого можно использовать метод `assertEquals()` из библиотеки JUnit.

Метод `assertEquals()` принимает два параметра - ожидаемое и фактическое значение. Если эти значения равны, утверждение и тест пройдут. Если они не равны, утверждение и тест проваливаются.

Вызовите этот метод `assertEquals()`, а затем передайте в качестве параметров переменные `expectedTip` и `actualTip`:

```
import org.junit.Assert.assertEquals
import org.junit.Test
import java.text.NumberFormat

class TipCalculatorTests {

    @Test
    fun calculateTip_20PercentNoRoundup() {
        val amount = 10.00
        val tipPercent = 20.00
        val expectedTip = NumberFormat.getCurrencyInstance().format(2)
        val actualTip = calculateTip(amount = amount, tipPercent = tipPercent,
false)
        assertEquals(expectedTip, actualTip)
    }
}
```

Примечание: В библиотеке JUnit существует множество утверждений. Некоторые распространенные утверждения, с которыми вы можете столкнуться, следующие:

- `assertEquals()`
- `assertNotEquals()`
- `assertTrue()`
- `assertFalse()`
- `assertNull()`
- `assertNotNull()`
- `assertThat()`

Для получения дополнительной информации смотрите раздел `Assert`.

Запуск теста Теперь пришло время запустить ваш тест!

Вы, наверное, заметили, что в желобе рядом с номером строки имени вашего класса и тестовой функции появились стрелки. Вы можете нажать на эти стрелки, чтобы запустить тест. Когда вы нажимаете стрелку рядом с методом, вы запускаете только этот тестовый метод. Если в классе есть несколько тестовых методов, вы можете щелкнуть стрелку рядом с классом, чтобы запустить все тестовые методы этого класса.

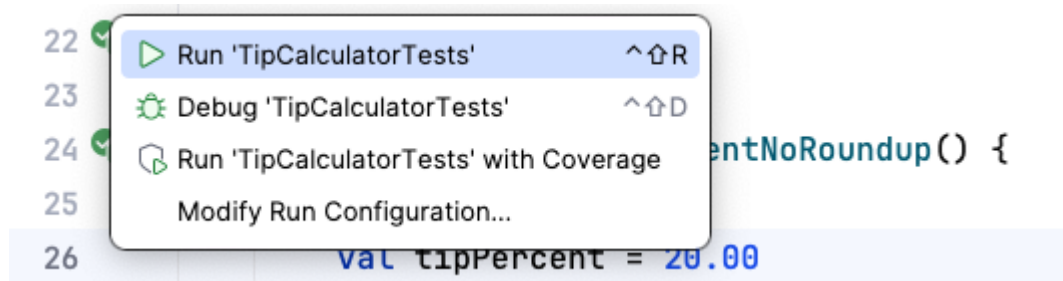
```

22  class TipCalculatorTests {
23      @Test
24      fun calculateTip_20PercentNoRoundup() {
25          val amount = 10.00

```

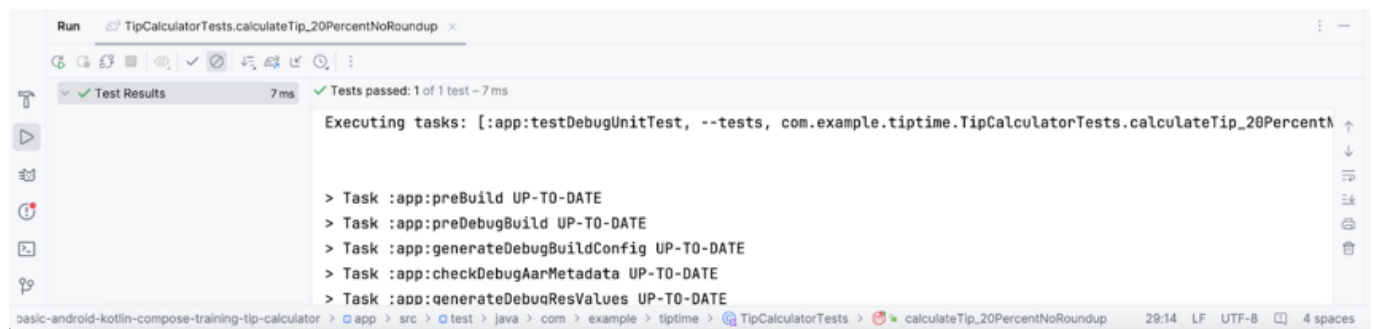
Запустите тест:

Нажмите на стрелки рядом с объявлением класса, а затем выберите Run 'TipCalculatorTests'.



Вы должны увидеть следующее:

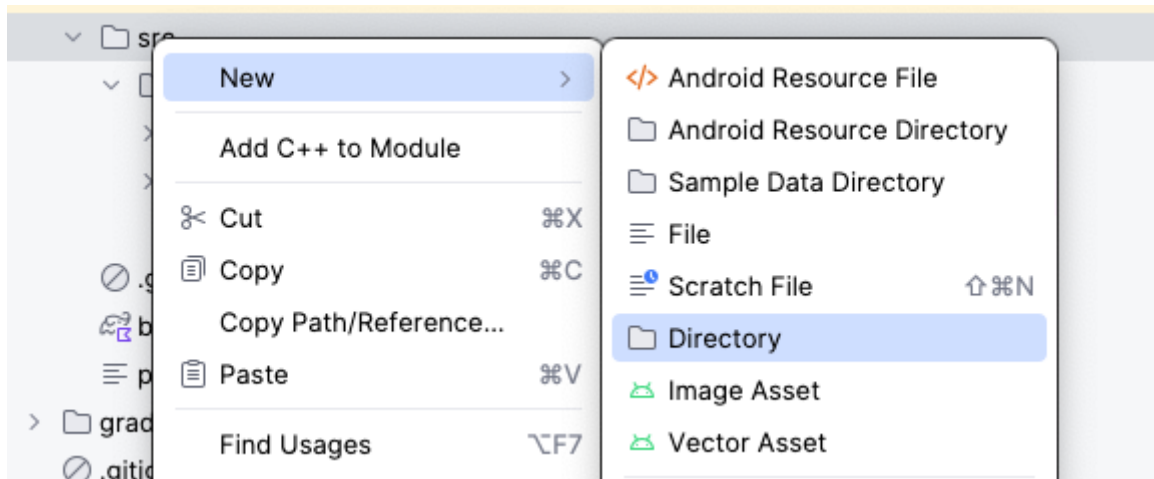
В нижней части панели «Выполнить» вы видите некоторые результаты.



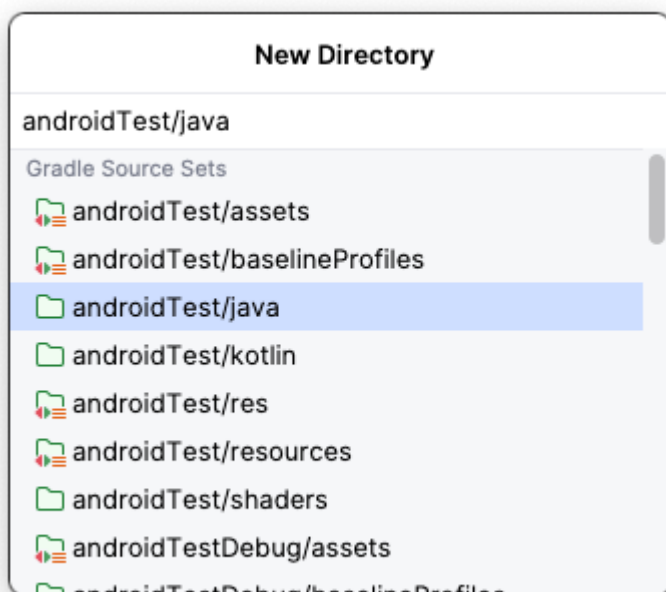
5. Напишите тест инструментария

Создайте каталог инструментария Каталог инструментария создается аналогично каталогу локального теста.

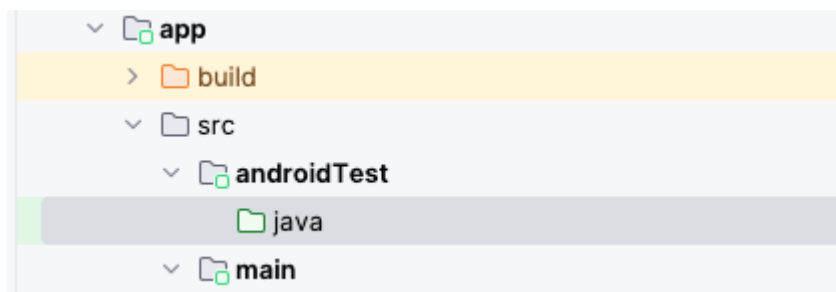
Щелкните правой кнопкой мыши на каталоге src и выберите New > Directory.



В окне New Directory выберите androidTest/java.



Наберите на клавиатуре клавишу return или enter. Теперь каталог androidTest можно увидеть на вкладке Project.



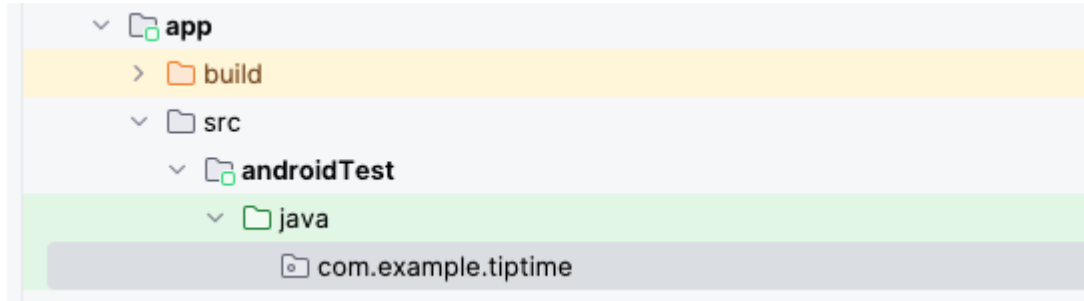
Подобно тому, как директории main и test имеют одинаковую структуру пакетов, директория androidTest должна содержать такую же структуру пакетов.

Щелкните правой кнопкой мыши на папке androidTest/java и выберите New > Package. В окне Новый пакет введите com.example.tiptime. Нажмите на клавиатуре клавишу return или enter. Теперь полная структура пакетов для каталога androidTest видна на вкладке Project. Создание тестового класса В проектах Android каталог для тестирования инструментария обозначается как каталог androidTest.

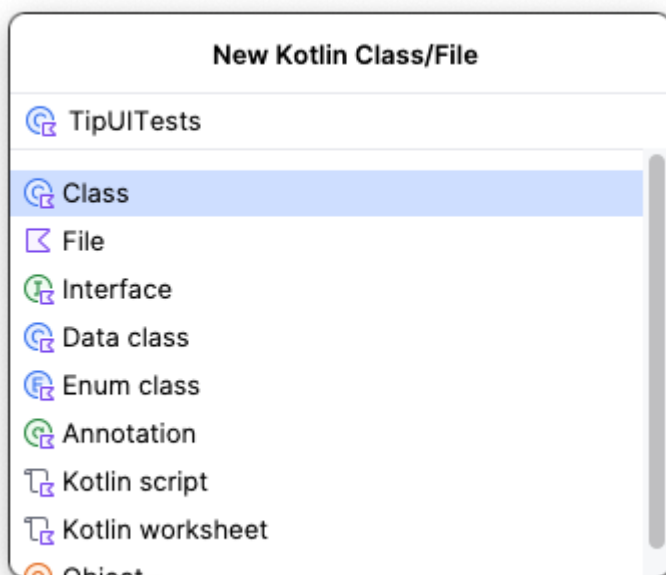
Чтобы создать инструментальный тест, нужно повторить тот же процесс, что и при создании локального теста, но на этот раз создать его в директории `androidTest`.

Создайте класс теста:

Перейдите в каталог `androidTest` на панели проекта. Щелкайте по стрелкам `cf54f6c094aa8fa3.png` рядом с каждым каталогом, пока не увидите каталог `tiptime`.



Щелкните правой кнопкой мыши каталог `tiptime`, а затем выберите `New > Kotlin Class/File`. В качестве имени класса введите `TipUITests`.



Напишите тест Код инструментальных тестов сильно отличается от кода локальных тестов.

Инструментальные тесты тестируют реальный экземпляр приложения и его пользовательский интерфейс, поэтому содержимое пользовательского интерфейса должно быть установлено, подобно тому, как содержимое устанавливается в методе `onCreate()` файла `MainActivity.kt`, когда вы писали код для приложения `Tip Time`. Это нужно сделать до того, как вы напишете все инструментальные тесты для приложений, созданных с помощью `Compose`.

В случае с тестами приложения `Tip Time` вы должны написать инструкции по взаимодействию с компонентами пользовательского интерфейса, чтобы процесс расчета чаевых тестировался через пользовательский интерфейс. Концепция инструментального теста может показаться абстрактной, но не волнуйтесь! Этот процесс рассматривается в следующих шагах.

Напишите тест:

Создайте переменную `composeTestRule`, установленную на результат метода `createComposeRule()`, и аннотируйте ее аннотацией `Rule`:

```
import androidx.compose.ui.test.junit4.createComposeRule
import org.junit.Rule

class TipUITests {

    @get:Rule
    val composeTestRule = createComposeRule()
}
```

Создайте метод `calculate_20_percent_tip()` и аннотируйте его с помощью аннотации `@Test`:

```
import org.junit.Test

@Test
fun calculate_20_percent_tip() {
}
```

Компилятор знает, что методы, аннотированные аннотацией `@Test` в каталоге `androidTest`, относятся к инструментальным тестам, а методы, аннотированные аннотацией `@Test` в каталоге `test`, относятся к локальным тестам.

В теле функции вызовите функцию `composeTestRule.setContent()`. Она устанавливает содержимое пользовательского интерфейса правила `composeTestRule`. В теле лямбды функции вызовите функцию `TipTimeTheme()` с телом лямбды, вызывающим функцию `TipTimeLayout()`.

```
import com.example.tiptime.ui.theme.TipTimeTheme

@Test
fun calculate_20_percent_tip() {
    composeTestRule.setContent {
        TipTimeTheme {
            TipTimeLayout()
        }
    }
}
```

По окончании работы код должен быть похож на код, написанный для установки содержимого в методе `onCreate()` в файле `MainActivity.kt`. Теперь, когда содержимое пользовательского интерфейса настроено, вы можете написать инструкции для взаимодействия с компонентами пользовательского интерфейса приложения. В этом приложении вам нужно проверить, что приложение отображает правильное значение чаевых на основе вводимых данных о сумме счета и проценте чаевых.

Доступ к компонентам пользовательского интерфейса можно получить в виде узлов через правило `composeTestRule`. Обычный способ сделать это - обратиться к узлу, содержащему определенный текст, с помощью метода `onNodeWithText()`. Используйте метод `onNodeWithText()` для доступа к текстовому полю, предназначенному для суммы счета:

```
import androidx.compose.ui.test.onNodeWithText

@Test
fun calculate_20_percent_tip() {
    composeTestRule.setContent {
        TipTimeTheme {
            TipTimeLayout()
        }
    }
    composeTestRule.onNodeWithText("Bill Amount")
}
```

Далее вы можете вызвать метод `performTextInput()` и передать в него текст, который вы хотите ввести для заполнения составного `TextField`.

Заполните текстовое поле для суммы счета значением 10:

```
import androidx.compose.ui.test.performTextInput

@Test
fun calculate_20_percent_tip() {
    composeTestRule.setContent {
        TipTimeTheme {
            TipTimeLayout()
        }
    }
    composeTestRule.onNodeWithText("Bill Amount")
        .performTextInput("10")
}
```

Используйте тот же подход, чтобы заполнить поле `OutlinedTextField` для процента чаевых значением 20:

```
@Test
fun calculate_20_percent_tip() {
    composeTestRule.setContent {
        TipTimeTheme {
            TipTimeLayout()
        }
    }
    composeTestRule.onNodeWithText("Bill Amount")
        .performTextInput("10")
    composeTestRule.onNodeWithText("Tip Percentage").performTextInput("20")
}
```

После заполнения всех составных элементов TextField подсказка отображается в составном элементе Text в нижней части экрана приложения.

Теперь, когда вы поручили тесту заполнить эти составные TextField, вам нужно убедиться, что составной Text отображает правильную подсказку с помощью утверждения.

В инструментальных тестах с Compose утверждения можно вызывать непосредственно на компонентах пользовательского интерфейса. Существует множество утверждений, но в данном случае вы хотите использовать метод `assertExists()`. Ожидается, что компонент Text, отображающий сумму чаевых, будет отображать: Сумма чаевых: \$2.00.

Примечание: символ валюты может меняться в зависимости от локали устройства. Использование символа '\$' является лишь примером. Валюта должна быть отформатирована в зависимости от локали.

Make an assertion that a node with that text exists:

```
import java.text.NumberFormat

@Test
fun calculate_20_percent_tip() {
    composeTestRule.setContent {
        TipTimeTheme {
            Surface (modifier = Modifier.fillMaxSize()){
                TipTimeLayout()
            }
        }
    }
    composeTestRule.onNodeWithText("Bill Amount")
        .performTextInput("10")
    composeTestRule.onNodeWithText("Tip Percentage").performTextInput("20")
    val expectedTip = NumberFormat.getCurrencyInstance().format(2)
    composeTestRule.onNodeWithText("Tip Amount: $expectedTip").assertExists(
        "No node with this text was found."
    )
}
```

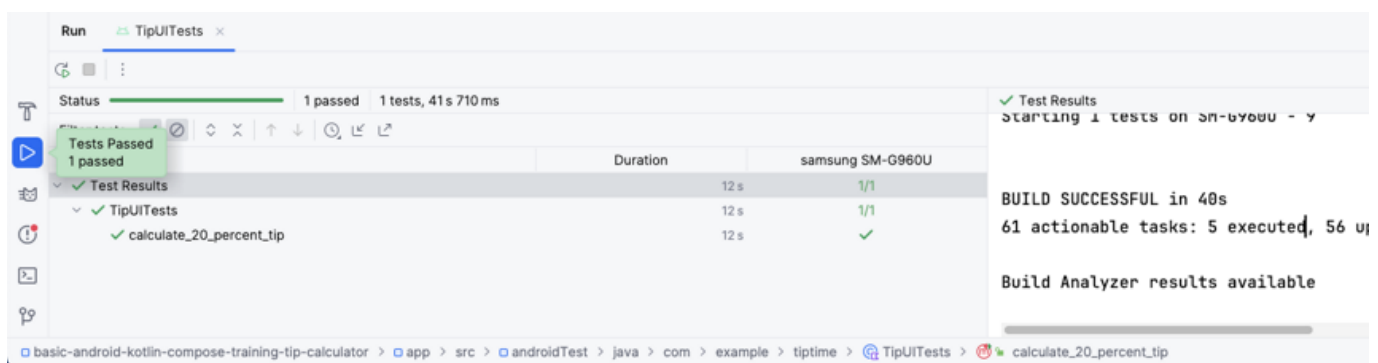
Проведите испытание Процесс запуска инструментального теста такой же, как и для локального теста. Чтобы запустить отдельный тест или весь класс тестов, нажмите на стрелки в желобе рядом с каждым объявлением.

```

29 >> class TipUITests {
30
31     @get:Rule
32     val composeTestRule = createComposeRule()
33
34     @Test
35     fun calculate_20_percent_tip() {
36         composeTestRule.setContent {
37             TipTimeTheme {

```

Нажмите на стрелки рядом с объявлением класса. Вы можете увидеть, как тесты выполняются на вашем устройстве или эмуляторе. По завершении теста вы должны увидеть вывод, показанный на этом изображении:



Заключение

Вы написали свои первые автоматизированные тесты в Android. Тесты - важнейший компонент контроля качества программного обеспечения. Продолжая создавать приложения для Android, убедитесь, что вы пишете тесты вместе с функциями ваших приложений, чтобы убедиться, что ваши приложения работают должным образом на протяжении всего процесса разработки.

Резюме

- Что такое автоматизированные тесты.
- Почему автоматизированные тесты важны.
- Разница между локальными и инструментальными тестами
- Основные лучшие практики написания автоматизированных тестов.
- Где найти и разместить классы локальных и инструментальных тестов в проекте Android.
- Как создать метод тестирования.
- Как создавать локальные и инструментальные тестовые классы.
- Как делать утверждения в локальных и инструментальных тестах.
- Как использовать правила тестирования.
- Как использовать ComposeTestRule для запуска приложения с тестом.
- Как взаимодействовать с композитами в инструментальных тестах.
- Как запускать тесты.